

Estruturas de Controle Condicionais

Em programação, é muito comum nos depararmos com situações em que uma parte do código só deve ser executada se determinada condição for satisfeita. Esse tipo de decisão é fundamental para que um programa possa se comportar de maneira inteligente e dinâmica.

Alguns exemplos comuns são:

- Verificar se o jogador atingiu a condição de vitória para encerrar um jogo;
- Impedir a execução de um cálculo inválido, como uma divisão por zero;
- Executar ações diferentes de acordo com a entrada do usuário;
- Tomar decisões com base em sensores, estados ou eventos do sistema.

Para lidar com esse tipo de situação, utilizamos as estruturas de controle condicionais, que permitem ao programa avaliar condições e escolher qual caminho seguir durante a execução.

Essas decisões são baseadas em expressões que resultam em valores do tipo booleano (bool). Uma variável booleana pode assumir apenas dois valores:

- `True` (verdadeiro)
- `False` (falso)

A partir dessas expressões booleanas, o Python disponibiliza diferentes estruturas condicionais, como:

- `if`
- `if else`
- `if elif else`
- `match case`

Nas próximas seções, veremos como cada uma dessas estruturas funciona, quando utilizá-las e quais são suas principais diferenças.

✓ IF

O `if` é, sem dúvida, uma das estruturas de controle mais utilizadas em programação. A estrutura `if` (do inglês “se”) permite que um bloco de código seja executado apenas quando uma determinada condição for verdadeira.

Essa condição deve ser uma expressão booleana, ou seja, uma expressão que resulte em `True` ou `False`.

Se a condição avaliada for True, o bloco de código associado ao if é executado. Caso contrário, o bloco é simplesmente ignorado, e o programa segue para a próxima instrução.

Veja o exemplo abaixo:

```
if True:
    print("A condição é verdadeira")

if False:
    print("Esta linha não será executada")
```

A condição é verdadeira

Nesse caso, utilizamos os próprios valores de `True` e `False` para ficar mais claro. Mas na programação real utilizamos os operadores lógicos e relacionais que vimos na aula 3. Revisando eles:

- Operadores lógicos

Operador	Nome	Descrição	Exemplo	Resultado
<code>and</code>	E lógico	Verdadeiro se todas as condições forem verdadeiras	<code>True and False</code>	<code>False</code>
<code>or</code>	OU lógico	Verdadeiro se pelo menos uma condição for verdadeira	<code>True or False</code>	<code>True</code>
<code>not</code>	NÃO lógico	Inverte o valor lógico	<code>not True</code>	<code>False</code>

- Operadores Relacionais

Operador	Nome	Descrição	Exemplo	Resultado
<code>==</code>	Igual a	Verifica se dois valores são iguais	<code>5 == 5</code>	<code>True</code>
<code>!=</code>	Diferente de	Verifica se dois valores são diferentes	<code>5 != 3</code>	<code>True</code>
<code>></code>	Maior que	Verifica se o valor da esquerda é maior	<code>5 > 3</code>	<code>True</code>
<code><</code>	Menor que	Verifica se o valor da esquerda é menor	<code>5 < 3</code>	<code>False</code>
<code>>=</code>	Maior ou igual a	Verifica se é maior ou igual	<code>5 >= 5</code>	<code>True</code>
<code><=</code>	Menor ou igual a	Verifica se é menor ou igual	<code>4 <= 6</code>	<code>True</code>

```
# Utilizando operadores relacionais
a = 10 ; b = 5

# Exemplo de condição
condicao = (a > b)
print("Tipo da condição:", type(condicao))

# Utilizando condição direta no if
if a > b:
    print("a é maior que b")
```

Tipo da condição: `<class 'bool'>`
a é maior que b

Podemos fazer um condição composta utilizando operadores lógicos como `and`, `or` e etc.

```
a = 10 ; b = 5 ; c = 15

if a > b and a < c:
    print("a é maior que b e menor que c")
```

a é maior que b e menor que c

```
a = 10 ; b = 5 ; c = 2

if a > b or a > c:
    print("a é maior que b ou c")
```

a é maior que b ou c

✓ IF ... ELSE

A estrutura `if ... else` pode ser interpretada como “se ... então ..., caso contrário ...”. Ela permite definir dois caminhos possíveis de execução, de acordo com o resultado de uma condição.

- Se a condição for verdadeira, o primeiro bloco de código é executado;
- Caso a condição seja falsa, o bloco de código associado ao else é executado.

Essa estrutura é especialmente útil porque substitui o uso de dois if com condições inversas, tornando o código mais claro, organizado e fácil de manter.

Veja o exemplo abaixo:

```
# Usando dois IFs separados
a = 10 ; b = 5

if a == b:
    print("a é igual a b")

if a != b:
    print("a é diferente de b")
```

a é diferente de b

```
if a == b:
    print("a é igual a b")
else:
    print("a é diferente de b")
```

a é diferente de b

✓ IF ... ELFI ... ELIF ... ELSE

A estrutura `if ... elif ... else` pode ser entendida como “se ... então, senão se ... então, caso contrário ...”. Ela permite avaliar múltiplas condições em sequência, executando apenas o primeiro bloco cuja condição seja verdadeira.

O funcionamento ocorre da seguinte forma:

- A condição do if é avaliada primeiro;
- Se for verdadeira, o bloco correspondente é executado e as demais condições são ignoradas;
- Caso seja falsa, a próxima condição (elif) é avaliada;
- Esse processo continua até que uma condição seja satisfeita;
- Se nenhuma condição for verdadeira, o bloco do else (se existir) é executado.

Essa estrutura é muito utilizada quando precisamos tomar decisões com base em comparações, geralmente envolvendo operadores relacionais.

Veja o exemplo abaixo:

```
if 10 > 20:
    print("10 é maior que 20")

if 10 > 15:
    print("10 é maior que 15")

if 10 > 5:
    print("10 é maior que 5")
```

10 é maior que 5

```
if 10 > 20:
    # Se a condição for verdadeira
    print("10 é maior que 20")

elif 10 < 15:
    # Se a primeira condição for falsa, verifica esta
    print("10 é menor que 15")

elif 10 > 5:
    # Se as condições anteriores forem falsas, verifica esta
    print("10 é maior que 5")
else:
    # Se nenhuma das condições anteriores for verdadeira (Opcional)
    print("10 é 10")
```

10 é maior que 5

✓ MATCH ... CASE

A estrutura `match ... case` é uma forma mais organizada e legível de lidar com decisões que envolvem várias possibilidades para uma mesma variável ou estrutura de dados. Ela foi introduzida no Python a partir da versão 3.10 e é baseada no conceito de pattern matching (casamento de padrões).

De forma intuitiva, o match pode ser entendido como:

“Avalie um valor e execute o bloco de código correspondente ao padrão que melhor se encaixa nele.”

Essa estrutura é especialmente útil quando, para resolver um problema, seria necessário escrever vários `if` e `elif` verificando a mesma variável, o que tornaria o código longo e repetitivo.

Por exemplo, ao invés de escrever:

```
if comando == "frente":  
    ...  
elif comando == "tras":  
    ...  
elif comando == "parar":  
    ...
```

Podemos utilizar:

```
match comando:  
    case "frente":  
        ...  
    case "tras":  
        ...  
    case "parar":  
        ...
```

Além de melhorar a legibilidade, o `match ... case` permite trabalhar não apenas com valores simples, mas também com estruturas mais complexas, como listas, tuplas e dicionários, possibilitando a desestruturação de dados diretamente na condição.

Outro ponto importante é que o Python avalia os case de cima para baixo, executando apenas o primeiro padrão que for compatível. Caso nenhum padrão seja satisfeito, o case `_` pode ser utilizado como um caso padrão (equivalente ao else).

Em resumo é indicado quando:

- Existem muitas condições baseadas no mesmo valor;
- O código com if/elif começa a perder legibilidade;
- Queremos expressar decisões de forma mais semântica e estruturada.

```
opcao = 2

match opcao:
    case 1:
        print("Opção 1 selecionada")
    case 2:
        print("Opção 2 selecionada")
    case 3:
        print("Opção 3 selecionada")
    case _:
        print("Opção inválida")
```

Opção 2 selecionada

```
comando = "parar"

match comando:
    case "frente":
        print("Movendo para frente")
    case "tras":
        print("Movendo para trás")
    case "parar":
        print("Robô parado")
    case _:
        print("Comando desconhecido")
```

Robô parado

Observação

Ainda há muito conteúdo sobre as estruturas de condição, principalmente quando trabalhamos com outras estruturas de dados. Veremos novamente elas quando estudarmos mais sobre essas estruturas de dados.

